Functions

Lec-13_14_15_16

Introduction

- Experience has shown that the best way to develop and maintain a large program is to construct it from smaller, more manageable pieces.
- This technique is called divide and conquer.
- Using existing functions as building blocks for creating new programs is a key aspect of software reusability, it’s also a major benefit of object-oriented programming.

Definition of Function

```
In [1]: def function_name(parameter_list):
   ...:     """Docstring that explains the function"""
   ...:     # function body
   ...:     return
   ...:

In [2]: def empty_function():
   ...:     pass
   ...:

In [3]:
```

```
In [1]: def square(number):
   ...:     """Calculate the square of number."""
   ...:     return number ** 2
   ...:

In [2]: square(7)
Out[2]: 49

In [3]: square(2.5)
Out[3]: 6.25
```

- A custom function can be defined using the def keyword.
- A docstring is used that describes the purpose of the function
- Details about the function can be seen in Ipython by writing
  <function_name?>
- Statements inside the function are written once and can be called as many times during the execution of program.

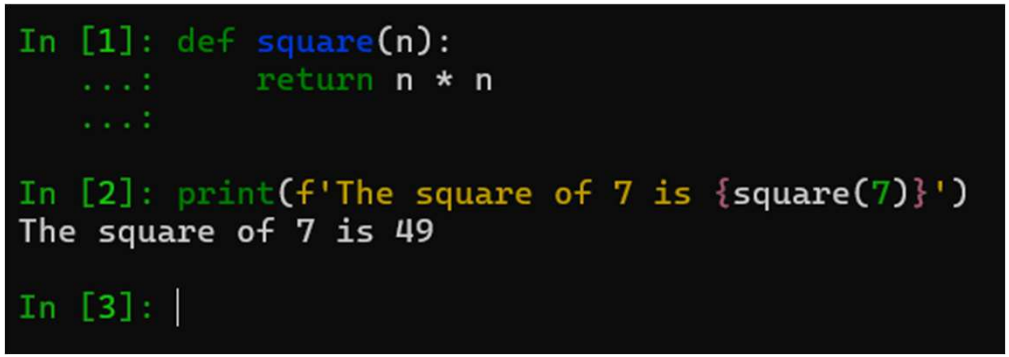
Definition of Function

- To call a function mention the function name and pass the parameters if any.
- While calling the function the number of arguments must match, otherwise the interpreter displays a “TypeError” message.
- Multiple parameters can be passed to the function inside the parenthesis by identifiers separated by comma (,).

```
In [4]: def pow_fun(base, power):
   ...:     return base ** power
   ...:

In [5]: pow_fun(2,3)
Out[5]: 8

In [6]: pow_fun(8,1/2)
Out[6]: 2.8284271247461903

In [7]: pow_fun(8,1/3)
Out[7]: 2.0
```

Returning result to function's caller

- A function in mathematics computes a value (and returns it). In a program it usually behaves in the same way.
- Usually because there are functions that don't return anything. Example print(f'Hello World') doesn't return anything.
- The return statement in a function is always the final statement in that function. Statements after the return statement are unreachable.



Returning a Result to a Function's Caller

- Executing a return statement without an expression terminates the function and implicitly returns the value None to the caller. The Python documentation states that None represents the absence of a value. None evaluates to False in conditions.
- When there's no return statement in a function, it implicitly returns the value None after executing the last statement in the function's block.

What happens during a function call?

- The default flow control is from top to bottom.
- The program is executed sequentially.
- When there is a function call, it goes to the location where the function is defined and executes the statements inside the function.
- Upon completion it comes back to the location of the call, and resumes sequential execution till the end of the program.

```
In [1]: def square(n):
   ...:     return n * n
   ...:

In [2]: print(f'The square of 7 is {square(7)}')
The square of 7 is 49

In [3]: print(1)
1

In [4]: print(2)
2

In [5]: print(3)
3

In [6]: square(4)
Out[6]: 16

In [7]:
```

Accessing a Function's Docstring via IPython's Help Mechanism

- Ipython has a mechanism that allows us to view the function's docstring and even source code too. (It is exclusive to Ipython only)
- function_name? to view the docstring, signature etc.
- function_name?? to view the above along with the source code

```
In [7]: square?
Signature: square(n)
Docstring: <no docstring>
File:      c:\users\ambar\<ipython-input-1-a2ebc361f67f>
Type:      function

In [8]: square??
Signature: square(n)
Docstring: <no docstring>
Source:
def square(n):
    return n * n
File:      c:\users\ambar\<ipython-input-1-a2ebc361f67f>
Type:      function
```

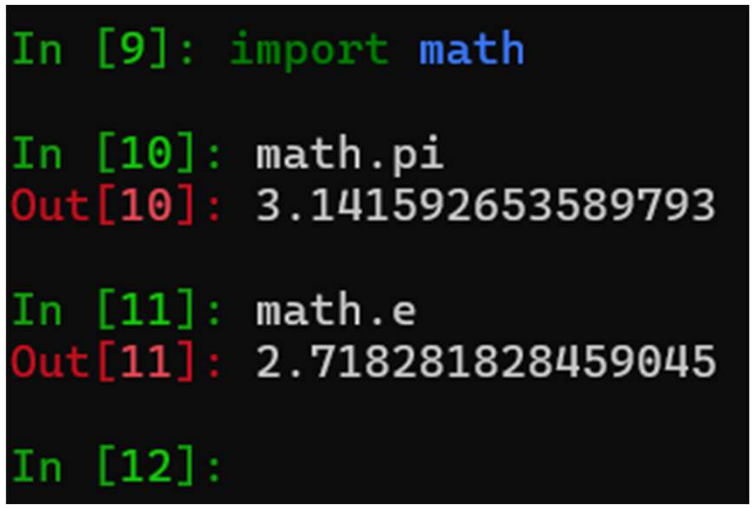
Python's Built-In max and min Functions

- Python has a built in max and min functions, that can be used directly by calling them.
- The functions can take one or multiple parameters as input and return the maximum and minimum value.

```
In [6]: max('yellow', 'red', 'orange', 'blue', 'green')
Out[6]: 'yellow'

In [7]: min(15, 9, 27, 14)
Out[7]: 9
```

Modules

- We have defined our own functions so far.
- Sometimes, we just want to directly use a function to carry out a task without trying to know about how it was implemented.
- This is realised with the help of modules.
- Python comes with built-in modules, that we can “import” as needed and use it.
- To use a function from that module the syntax is module_name.function_name()



Random-Number Generation

- In data science, the requirement of generating random numbers is very likely.
- To generate random numbers first, we need to import the random module.
- Inside the random module, there is a function called randrange(), where we can pass two parameters, the range which is in the form of [x,y).
- This example tries to simulate a dice roll.

```
In [1]: import random

In [2]: for roll in range(10):
   ...:     print(random.randrange(1, 7), end=' ')
   ...:
4 2 5 5 4 6 4 6 1 5
```

Random Numbers are not really random

- Function randrange() actually generates pseudorandom numbers, based on an internal calculation that begins with a numeric value known as a “seed”.
- This is provided so that it helps us to reproduce errors before we start to debug it.

```
import random
random.seed(10)
for roll in range(1,10):
    print(random.randrange(1,7), end=' ')


5 1 4 4 5 1 2 4 4
random.seed(11)
for roll in range(1,10):
    print(random.randrange(1,7), end=' ')


4 5 4 4 5 5 2 2 5
random.seed(10)
for roll in range(1,10):
    print(random.randrange(1,7), end=' ')


5 1 4 4 5 1 2 4 4
for roll in range(1,10):
    print(random.randrange(1,7), end=' ')


3 6 2 1 5 4 3 1 2
```

Python Standard Library

- Programs are usually written by combining both functions and classes.
- We earlier discussed modules in brief.
- A module is a file that groups related functions, data and classes.
- A package is a term that groups related modules together.
- In script mode, every .py file we create is considered to be a module and can be imported as needed.

Python Standard Library

Some popular Python Standard Library modules

`collections`—Data structures beyond lists, tuples, dictionaries and sets.
Cryptography modules—Encrypting data for secure transmission.
`csv`—Processing comma-separated value files (like those in Excel).
`datetime`—Date and time manipulations. Also modules `time` and `calendar`.
`decimal`—Fixed-point and floating-point arithmetic, including monetary calculations.
`doctest`—Embed validation tests and expected results in docstrings for simple unit testing.
`gettext` and `locale`—Internationalization and localization modules.
`json`—JavaScript Object Notation (JSON) processing used with web services and NoSQL document databases.
`math`—Common math constants and operations.
`os`—Interacting with the operating system.
`profile`, `pstats`, `timeit`—Performance analysis.
`random`—Pseudorandom numbers.
`re`—Regular expressions for pattern matching.
`sqlite3`—SQLite relational database access.
`statistics`—Mathematical statistics functions such as `mean`, `median`, `mode` and `variance`.
`string`—String processing.
`sys`—Command-line argument processing; standard input, standard output and standard error streams.
`tkinter`—Graphical user interfaces (GUIs) and canvas-based graphics.
`turtle`—Turtle graphics.
`webbrowser`—For conveniently displaying web pages in Python apps.

math Module Functions

- The math module defines functions for performing various common mathematical calculations.

```
In [2]: math.sqrt(900)
Out[2]: 30.0
```

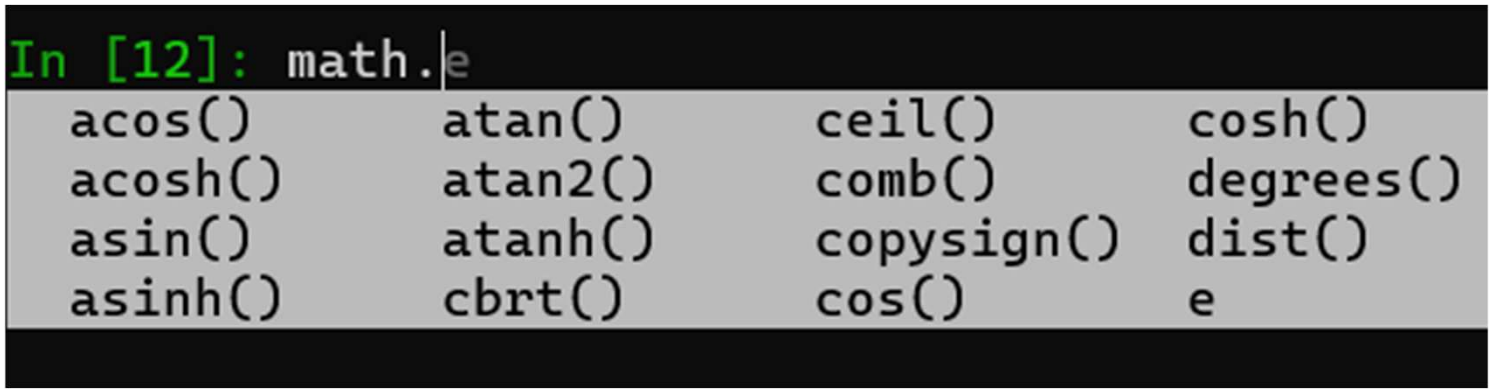
```
In [3]: math.fabs(-10)
Out[3]: 10.0
```

math Module Functions

Function	Description	Example
`ceil(`*x*`)`	Rounds *x* to the smallest integer not less than *x*	`ceil(9.2)` is `10.0` `ceil(-9.8)` is `-9.0`
`floor(`*x*`)`	Rounds *x* to the largest integer not greater than *x*	`floor(9.2)` is `9.0` `floor(-9.8)` is `-10.0`
`sin(`*x*`)`	Trigonometric sine of *x* (*x* in radians)	`sin(0.0)` is `0.0`
`cos(`*x*`)`	Trigonometric cosine of *x* (*x* in radians)	`cos(0.0)` is `1.0`
`tan(`*x*`)`	Trigonometric tangent of *x* (*x* in radians)	`tan(0.0)` is `0.0`
`exp(`*x*`)`	Exponential function e^x	`exp(1.0)` is `2.718282` `exp(2.0)` is `7.389056`
`log(`*x*`)`	Natural logarithm of *x* (base *e*)	`log(2.718282)` is `1.0` `log(7.389056)` is `2.0`
`log10(`*x*`)`	Logarithm of *x* (base 10)	`log10(10.0)` is `1.0` `log10(100.0)` is `2.0`
`pow(`*x*`, `*y*`)`	*x* raised to power *y* (x^y)	`pow(2.0, 7.0)` is `128.0` `pow(9.0, .5)` is `3.0`
`sqrt(`*x*`)`	square root of *x*	`sqrt(900.0)` is `30.0` `sqrt(9.0)` is `3.0`
`fabs(`*x*`)`	Absolute value of *x*—always returns a float. Python also has the built-in function `abs`, which returns an `int` or a `float`, based on its argument.	`fabs(5.1)` is `5.1` `fabs(-5.1)` is `5.1`
`fmod(`*x*`, `*y*`)`	Remainder of *x*/*y* as a floating-point number	`fmod(9.8, 4.0)` is `1.8`

Using IPython Tab Completion for Discovery

- In Ipython we can type in module_name. and then press tab to see the list of functions and values contained within the module.
- Python doesn't contain constants, we can make constants by naming our variables with all caps to denote a constant.



Default Values

- When defining a function, you can specify that a parameter has a default parameter value.
- When calling the function, if you omit the argument for a parameter with a default parameter value, the default value for that parameter is automatically passed.

```
In [1]: def rectangle_area(length=2, width=3):
   ...:     """Return a rectangle's area."""
   ...:     return length * width
   ...:
```

```
In [2]: rectangle_area()
Out[2]: 6
```

```
In [3]: rectangle_area(10)
Out[3]: 30
```

```
In [4]: rectangle_area(10, 5)
Out[4]: 50
```

Keyword Arguments

- When calling functions, you can use keyword arguments to pass arguments in any order.
- In each function call, you must place keyword arguments after a function's positional arguments—that is, any arguments for which you do not specify the parameter name.

```
def p(x, y):
    print(f'x : {x}, y : {y}')


p(y=4, x=2)
x : 2, y : 4
p(4, 2)
x : 4, y : 2
```

```
def f(a, b, c, d):
    print(f'a : {a}, b : {b}, c : {c}, d : {d}')


f(d=4, 1, 2, 3)
SyntaxError: positional argument follows keyword argument
f(1, 2, d=3, 4)
SyntaxError: positional argument follows keyword argument
f(1, 2, 3, b = 4)
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    f(1, 2, 3, b = 4)
TypeError: f() got multiple values for argument 'b'
f(1,2,3,d=4)
a : 1, b : 2, c : 3, d : 4
f(1,2,c=3, 4)
SyntaxError: positional argument follows keyword argument
f(1,2,4,c=3)
Traceback (most recent call last):
  File "<pyshell#13>", line 1, in <module>
    f(1,2,4,c=3)
TypeError: f() got multiple values for argument 'c'
```

Arbitrary Argument Lists

- Functions like max and min, can take on any number of arguments.
- The functions which we have created so far can take at most x variables as per our definition, with the help of default values.
- In real world, data is dynamic and sometimes, we might not know how many variables we would need to hold that incoming data.
- That's when *v comes into play.
- A user-defined function with *v can take variable number of arguments.

```
def average(*v):
    return sum(v)/len(v)

average(1,2,3,4,5)
3.0
average(1,2,3,4)
2.5
average(20)
20.0
average(20,30)
25.0
```

Arbitrary Argument Lists

- It can be used in a creative way, where the two args are v1, v2 (assume) and the third argument is *v which takes all the remaining values when passed to it in the form of a tuple (In chapter 5).

```
def c(v1, v2, *v):
    print(f'v1 : {v1}, v2 : {v2}, *v : {v}')


c(1,2)
v1 : 1, v2 : 2, *v : ()
c(1,2,3,4,5,6,7)
v1 : 1, v2 : 2, *v : (3, 4, 5, 6, 7)
```

Methods

- A method is simply a function that you call on an object using the form <object_name.method_name(arguments)>

```
In [1]: s = 'Hello'

In [2]: s.lower()  # call lower method on string object s
Out[2]: 'hello'

In [3]: s.upper()
Out[3]: 'HELLO'

In [4]: s
Out[4]: 'Hello'
```

Scope Rules

- A local variable's identifier has local scope. It's "in scope" only from its definition to the end of the function's block. It "goes out of scope" when the function returns to its caller. So, a local variable can be used only inside the function that defines it.
- Identifiers defined outside any function (or class) have global scope—these may include functions, variables and classes. Variables with global scope are known as global variables. Identifiers with global scope can be used in a .py file or interactive session anywhere after they're defined.

```
x = 400 # global scope

def f():
    c = 100 # local to f()


print(x)
400
print(c)
Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    print(c)
NameError: name 'c' is not defined
```

Accessing a Global Variable from a Function

```
In [1]: x = 7

In [2]: def access_global():
   ...:     print('x printed from access_global:', x)
   ...:

In [3]: access_global()
x printed from access_global: 7
```

```
In [4]: def try_to_modify_global():
   ...:     x = 3.5
   ...:     print('x printed from try_to_modify_global:', x)
   ...:

In [5]: try_to_modify_global()
x printed from try_to_modify_global: 3.5

In [6]: x
Out[6]: 7
```

Accessing a Global Variable from a Function

```
In [7]: def modify_global():
   ...:     global x
   ...:     x = 'hello'
   ...:     print('x printed from modify_global:', x)
   ...:

In [8]: modify_global()
x printed from modify_global: hello

In [9]: x
Out[9]: 'hello'
```

Blocks vs Suites

- We have written function blocks and control suites.
- If a variable is created in a block it is local to that block.
- If a variable is created in a control suite, the scope of that variable is dependent on where the control suite is
- If the control statement is in the global scope, then any variables defined in the control statement have global scope.
- If the control statement is in a function's block, then any variables defined in the control statement have local scope.

```
x = 0
if x == 0:
    don = 1


print(don)
1
```

Shadowing Functions

- In python it is allowed to use identifiers as variables which are built-in methods.
- Once they are used as variables they lose their method like properties.
- Now using that variable as a function will create an error.
- The built-in function got “shadowed”.

```
sum([1,2,3,4])
10

sum = 10 + 15
sum
25

sum([1,2,3,4])
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    sum([1,2,3,4])
TypeError: 'int' object is not callable
```

import: A Deeper Look

- Import is used to bring in modules that are not available directly (like math).
- To import a module the syntax is import module_name
- To import a specific function(s) the syntax is

  from module_name import x, y, z, …
- There exist a method to import all the functions of the module.

  from module_name import *
- Sometimes a module name is too long and we want to make it short for frequent usage. The 'as' clause helps us to import the module and use it with a shorter name.

  import module_name as mn

```
In [1]: from math import ceil, floor

In [2]: ceil(10.3)
Out[2]: 11

In [3]: floor(10.7)
Out[3]: 10
```

```
In [4]: e = 'hello'

In [5]: from math import *

In [6]: e
Out[6]: 2.718281828459045
```

```
In [7]: import statistics as stats

In [8]: grades = [85, 93, 45, 87, 93]

In [9]: stats.mean(grades)
Out[9]: 80.6
```

Passing arguments to functions

- In programming languages, there are usually 2 ways to pass arguments.
- With pass-by-value, the called function receives a copy of the argument's value and works exclusively with that copy. Changes to the function's copy do not affect the original variable's value in the caller.
- With pass-by-reference, the called function can access the argument's value in the caller directly and modify the value if it's mutable.
- In python, arguments are always passed by reference (even functions too).

Passing arguments to functions

- A statement like x = 7 when executed, it doesn't mean, 7 is stored in x.
- In python it means, that there is an object created in the memory where 7 is stored. x references to the object where 7 is stored.
- It can be verified using id().
- No two objects in memory can have the same identity.

```
x = 7
id(x)
140731786709624
```

Passing arguments to functions

- ID of global variable ‘x’ and local variable ‘n’, local to square() is identical.
- Which proves that in python values are passed by reference.
- What will happen if we modify ‘n’?

```
x = 7
id(x)
140731786709624


def square(n):
    print(id(n))
    return n * n

square(x)
140731786709624
49
```

Passing arguments to functions

- We can see that before modifications the IDs were identical.
- Upon assignment we see that the IDs are different (last 3 digits).
- It also shows that, ints are immutable.
- Changing an immutable object creates a new object.

```
def modify_n(n):
    print(id(n))
    n = 14
    print(id(n))
```

```
id(x)
140731786709624
modify_n(x)
140731786709624
140731786709848
x
7
```

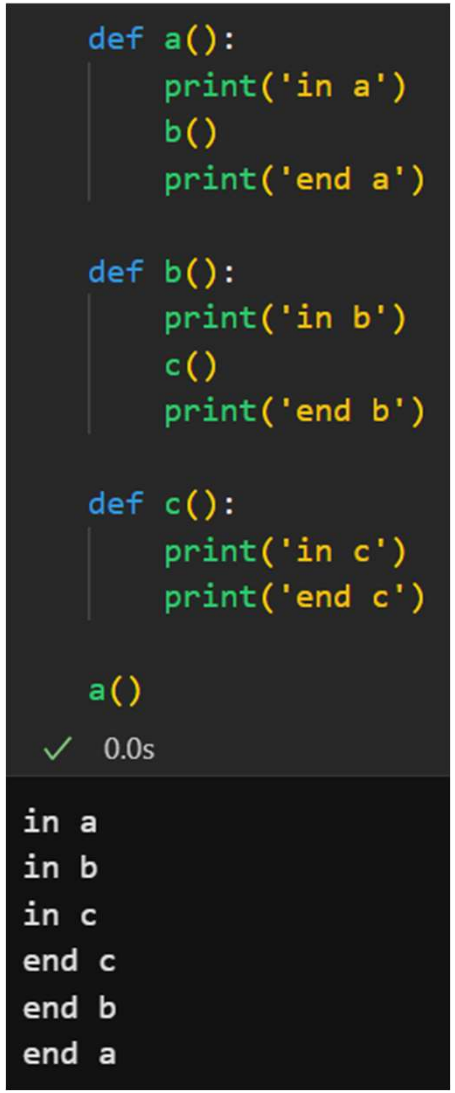
Passing arguments to functions

- Two objects if they are identical are said to be pointing to the same object.
- It can be verified using the ‘is’ operator.

```
In [5]: def cube(number):
   ...:     print('number is x:', number is x)
   ...:     return number ** 3
   ...:

In [6]: cube(x)
number is x: True
Out[6]: 343
```

Function Call Stack

- How python performs function calls?
- How does it remember to know to which function return to?
- The answer is stack frames.
- For each function call, the interpreter pushes an entry called a stack frame (or an activation record) onto the stack. This entry contains the return location that the called function needs so it can return control to its caller. When the function finishes executing, the inter preter pops the function's stack frame, and control transfers to the return location that was popped.



Local Variables and Stack Frames

- Each activation record, contains a detail of the variables local to the function along with the point of return.
- The local variables must satisfy these 3 conditions.
- They must exist while the function is running.
- They must remain active if the function makes a call to another function.
- Must be garbage collected once the function has completed its execution.

Stack Overflow

- Memory is finite, so in the function-call stack a limited number of functions can reside simultaneously.
- If this limit is reached, attempting to place another activation record will cause a stack overflow error.
- It usually happens with a faulty logic, where a function keeps calling other functions without ever completing its execution.

Principle of Least Privilege

- The principle of least privilege is fundamental to good software engineering.
- It states that code should be granted only the amount of privilege and access that it needs to accomplish its designated task, but no more.
- This is why a function's local variables are placed in stack frames on the function-call stack, so they can be used by that function while it executes and go away when it returns. Once the stack frame is popped, the memory that was occupied by it can be reused for new stack frames. Also, there is no access between stack frames, so functions cannot see each other's local variables.

Functional-Style Programming

- Python is an object oriented language, but it still offers functional style features.
- Functional style programming has the benefit of parallelization.
- In functional style of programming the programmer’s focus is on what to do.
- How to achieve is usually abstracted from the programmer.
- Python’s rich library handles the “how”.
- Example : the for loop written with range.
- We specify how much to iterate. How it iterates is left to the language

Pure Functions

- A pure function's results depends only on the argument(s) you pass into it.
- A pure function is also deterministic.
- A pure function doesn't have side-effects (It doesn't modify the arguments passed).

```
In [1]: values = [1, 2, 3]

In [2]: sum(values)
Out[2]: 6

In [3]: sum(values)  # same call always returns same result
Out[3]: 6

In [4]: values
Out[5]: [1, 2, 3]
```

Intro to Data Science : Measures of Dispersion

- Population Variance : is calculated by the following steps.

    1 : Find the mean.

    2 : Subtract each data-point with the mean.

    3 : Square each data-point

    4 : Calculate the mean of squares.

- This helps in detecting outliers, the values that are farthest from the mean found in step 4.

Intro to Data Science : Measures of Dispersion

```
In [1]: import statistics

In [2]: statistics.pvariance([1, 3, 4, 2, 6, 5, 3, 4, 5, 2])
Out[2]: 2.25
```

Standard Deviation: It is the square-root of variance.

```
In [4]: import math

In [5]: math.sqrt(statistics.pvariance([1, 3, 4, 2, 6, 5, 3, 4, 5, 2]))
Out[5]: 1.5
```